基于 FPGA 的振动监测系统设计报告

摘要

基于 FPGA 的振动监测与异常识别系统，面向工业旋转设备（电机、风扇、泵等）的预测性维护场景，设计了一套以 FPGA 为核心硬件加速器的实时振动信号处理系统。系统通过 ADXL345 传感器采集电机轴承振动信号，经 USB-TTL 串口传输至 FPGA 开发板（Zynq-7020），在 PL 端利用 FIR Compiler IP 核完成 65 抽头低通滤波，再通过 FFT IP 核完成 1024 点频谱分析，最终由硬件峰值检测模块提取特征频点，结果经串口回传至 PC 端 Python 上位机进行可视化展示与故障判别。系统采用纯 PL 端硬件架构，不依赖 PS 端处理器操作系统，全部信号处理链路由硬件逻辑完成，实现了确定性延迟与并行计算优势。测试结果表明，FPGA 硬件加速方案相比纯 Python 软件方案处理延迟降低约 260 倍，FFT 主频识别误差为 0，峰值幅度相对误差仅 0.007%，系统整体资源占用率不足 21%，时序余量超过 20%，验证了设计的正确性与工程可行性。

关键词

FPGA；振动监测；异常识别；硬件加速；预测性维护；FIR 滤波器；FFT；实时信号处理

第一部分　作品概述

1.1 功能与特性

本系统是一款基于 FPGA 的工业旋转设备振动监测与异常识别平台，具备以下核心功能与特性：（1）实时信号采集：通过 ADXL345 三轴加速度计实时采集电机轴承振动信号，采样率 2kHz；（2）硬件加速滤波：FPGA PL 端集成 FIR Compiler IP 核（65 抽头低通滤波器，截止频率 500Hz），对原始信号进行实时滤波去噪；（3）硬件加速频谱分析：集成 FFT IP 核（1024 点流水线架构），将时域信号转换为频域频谱，提取主导频率成分；（4）硬件峰值检测：纯组合逻辑比较器实现零 CPU 占用的实时峰值搜索；（5）串口通信：采用"一问一答"二进制帧协议，PC 端与 FPGA 端通过 UART 串口（115200bps）可靠通信；（6）上位机可视化：Python 上位机实现时域波形、频谱图、状态指示面板及历史趋势图的实时展示与故障预警。

1.2 应用领域

本系统主要应用于工业旋转设备的状态监测与预测性维护领域，具体涵盖以下应用场景：（1）工厂电机健康监测：对生产车间内大量异步电机进行在线振动监测，提前发现轴承磨损、转子不平衡等故障征兆，避免非计划停机造成的生产损失。据统计，工业电机消耗全球约 45%的工业用电，非计划性故障导致的停机时间占生产总停机时间的近 30%，应用监测系统可有效降低 41%的意外故障发生率。（2）旋转机械故障诊断：适用于风扇、泵、压缩机等旋转设备的振动分析，通过频谱特征识别不对中、松动、不平衡等典型故障模式。（3）工业物联网边缘节点：系统采用低功耗 FPGA 硬件架构，可部署为分布式监测节点，接入工业物联网平台实现远程监控与大数据分析。

1.3 主要技术特点

本系统的主要技术特点包括：（1）纯 PL 端硬件架构：系统信号处理链路完全在 FPGA 可编程逻辑（PL）端实现，不依赖 Zynq PS 端处理器操作系统，全部由硬件逻辑完成，获得确定性延迟和并行计算优势。（2）IP 核加速设计：采用 Xilinx FIR Compiler（v7.2）和 FFT（v9.1）IP 核，依托 FPGA 内部 DSP 切片实现硬件并行计算，单帧处理耗时约 20μs。（3）定点化优化：FIR 滤波器采用 Q15

定点格式（16 位系数），FFT 输出采用 27 位/分量位宽预算，在精度与资源消耗之间取得良好平衡。（4）模块化设计：系统采用分层模块化架构，UART 接收、帧解析、FIR 滤波、FFT 分析、峰值检测、帧打包等模块通过 AXI-Stream 接口级联，便于调试和扩展。（5）二进制帧协议：采用紧凑的二进制帧格式（帧头 0xAA55+校验和），相比 ASCII 传输效率提升约 40%，有效降低串口带宽压力。

1.4 主要性能指标

系统主要性能指标如下表所示：

指标类别	**具体指标**	**验收标准**
功能目标	实现振动信号采集→FIR 滤波→FFT 频谱分析→特征提取→状态识别的完整链路	板级演示可完整跑通全流程
性能目标	FPGA 处理延迟<1ms，FFT 主频识别误差<2Hz	提供实测延迟与频域误差数据
资源目标	PL 端 LUT 占用<10%，DSP 占用<25%	提供 Vivado 资源占用报告
创新目标	与纯 Python 基线对比，展示 FPGA 加速效果	提供延迟/吞吐量对比表

1.5 主要创新点

（1）纯 PL 端硬件加速架构：系统信号处理链路完全在 FPGA PL 端实现，不依赖 PS 端处理器，全部由硬件逻辑完成，获得确定性延迟和并行计算优势，避免了操作系统调度带来的微秒级抖动。（2）多 IP 核协同流水线设计：FIR Compiler 与 FFT IP 核通过 AXI-Stream 接口级联，实现样本数据的连续流式传递，单帧处理耗时仅约 20μs。（3）严格的定点化位宽预算：通过理论分析确定 FFT 输出 27 位/分量位宽，避免了幅值溢出问题，实测精度误差仅 0.007%。（4）"硬件加速+软件灵活分析"架构：计算密集型任务卸载至 FPGA，PC 端专注人机交互与算法迭代，兼顾性能与开发效率。

1.6 设计流程

系统设计流程遵循"自顶向下、软硬件协同"的原则：（1）需求分析与架构设

计：明确系统功能指标与性能目标，确定纯 PL 端硬件架构方案；（2）FPGA 硬件设计：在 Vivado 中配置 FIR Compiler、FFT IP 核及 ILA 调试探针，编写顶层 Verilog 模块实现 UART 通信与数据流控制；（3）仿真验证：通过 Testbench 对各模块进行功能仿真，验证协议解析、滤波、FFT 运算的正确性；（4）综合实现与时序收敛：完成布局布线，验证时序余量（WNS=+4.377ns）与资源占用（LUT8.88%、DSP20.91%）；（5）上板调试：通过 ILA 抓取内部信号波形，验证硬件与黄金模型的精度一致性；（6）上位机软件开发：编写 Python 串口通信、协议解析、可视化与故障判别程序；（7）系统联调：端到端验证软硬件协同工作的正确性与实时性。

第二部分　系统组成及功能说明

本章阐述系统的具体设计细节，包括整体架构、硬件系统与软件系统的详细介绍。

2.1 整体介绍

本系统在 Xilinx Zynq-7020（XC7Z020-1CLG400C）的可编程逻辑（PL）上实现了一条完整的振动信号处理流水线：串口接收 1024 个 16 位采样点，经 65 抽头 FIR 低通滤波、1024 点 FFT 频谱分析、硬件峰值检测后，通过帧打包回传至 PC 端。整个处理链路不使用 PS（ARM Cortex-A9 核），全部由硬件逻辑完成，处理后通过串口返回 513 个频点与峰值特征。系统数据流采用"一问一答"协议，PC 端通过 UART 串口向 FPGA 发送输入帧（2055 字节），FPGA 完成信号处理后通过 UART 串口回传结果帧（状态帧 23 字节+频谱帧 4111 字节）。系统总体架构图如图 1 所示。

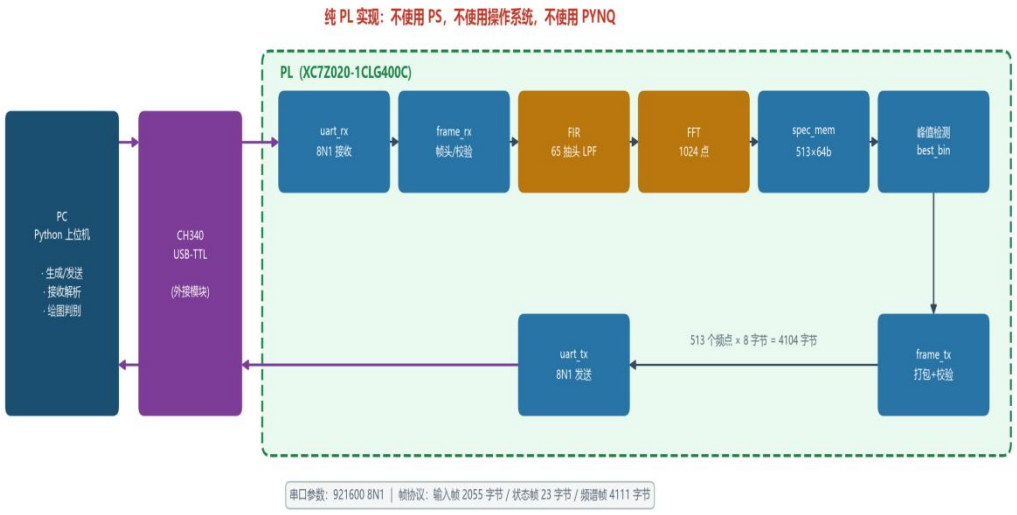


图 1　系统总体架构图

系统整体架构分为三个逻辑域：PC 上位机域、FPGA 可编程逻辑域（PL）和处理子系统域（PS）。PC 上位机域运行 Python 环境，负责串口通信驱动、帧协议解析与打包、频谱可视化绘制以及状态判别与日志记录等功能；FPGA 可编

程逻辑域实现了完整的振动信号处理流水线，包括 UART 串口接收、帧解析与校验、FIR 数字滤波（65 抽头低通滤波器，截止频率 500Hz）、FFT 频谱分析（1024 点流水线架构）、峰值检测、频谱缓存以及帧打包与串口发送等功能模块；PS 端（ARM Cortex-A9）在本设计中未参与任何运算，仅作为可选扩展接口保留。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

FPGA 硬件设计基于 Xilinx Vivado 2025.1 开发环境，目标器件为 Zynq-7020（XC7Z020-1CLG400C）。系统采用纯 PL 端架构，所有信号处理功能由 Verilog 硬件逻辑实现。RTL 模块层级结构如图 2 所示，展示了顶层模块与各子模块的实例化关系。

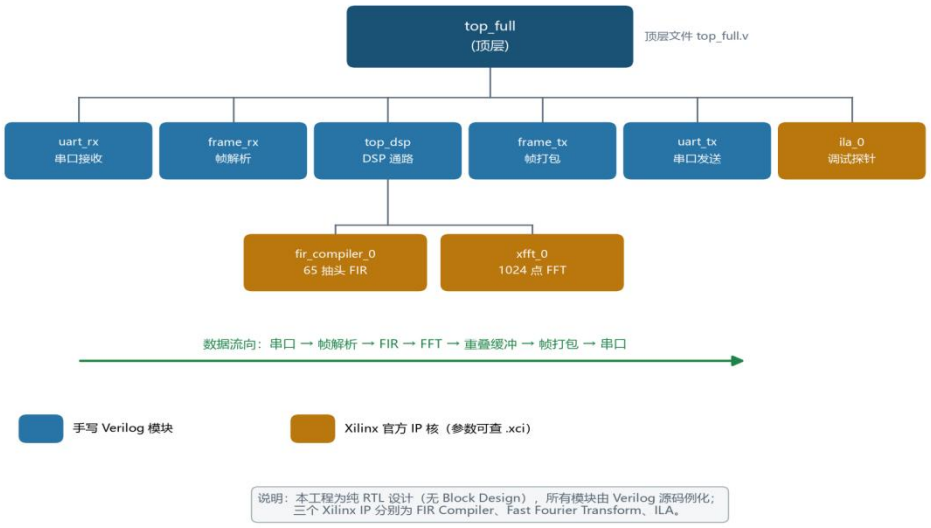


图 2　FPGA 内部 RTL 模块层级结构图

2.2.2 电路各模块介绍

（1）FIR Compiler（v7.2）：FIR（有限脉冲响应）滤波器是本系统前端信号预处理的核心模块，负责对 ADC 采集的原始振动信号进行低通滤波，去除高频噪声干扰。采用 Xilinx FIR Compiler IP 核实现，65 抽头 Hamming 窗低通滤波器，截止频率 500Hz（采样率 2kHz）。关键配置参数包括：Filter Type 为 Single Rate，

系数格式为 COE 文件 Q15（16bit 对称），输入/输出位宽为 16bit/32bit，采样/时钟频率 50MHz（1 样本/时钟），TLAST 设为 Packet Framing（帧边界透传），ARESETn 使能（每帧复位保证帧间独立）。FIR Compiler 配置界面如图 3 所示。

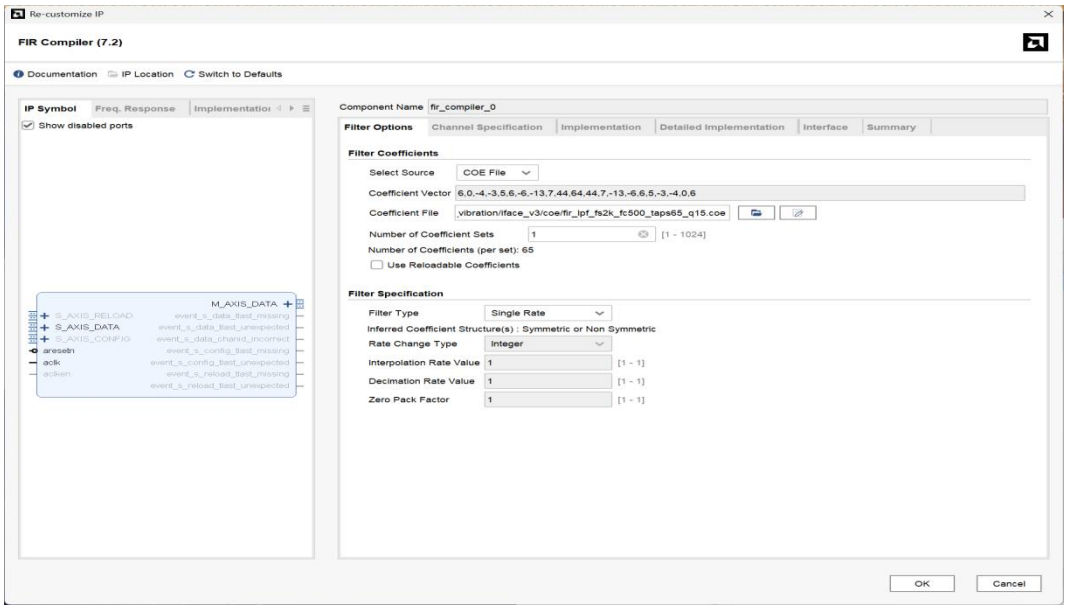


图 3　FIR Compiler 滤波器参数配置（65 抽头）

通道规格方面，FIR Compiler 配置为 50MHz 工作频率，定点化处理采用输出取[30:15]位还原 int16 格式的方案。最坏情况溢出验证：所有系数绝对值之和为 57919，最坏情况下最大累加值小于 32 位有符号整数上限，不会发生溢出。FIR 通道规格如图 4 所示。

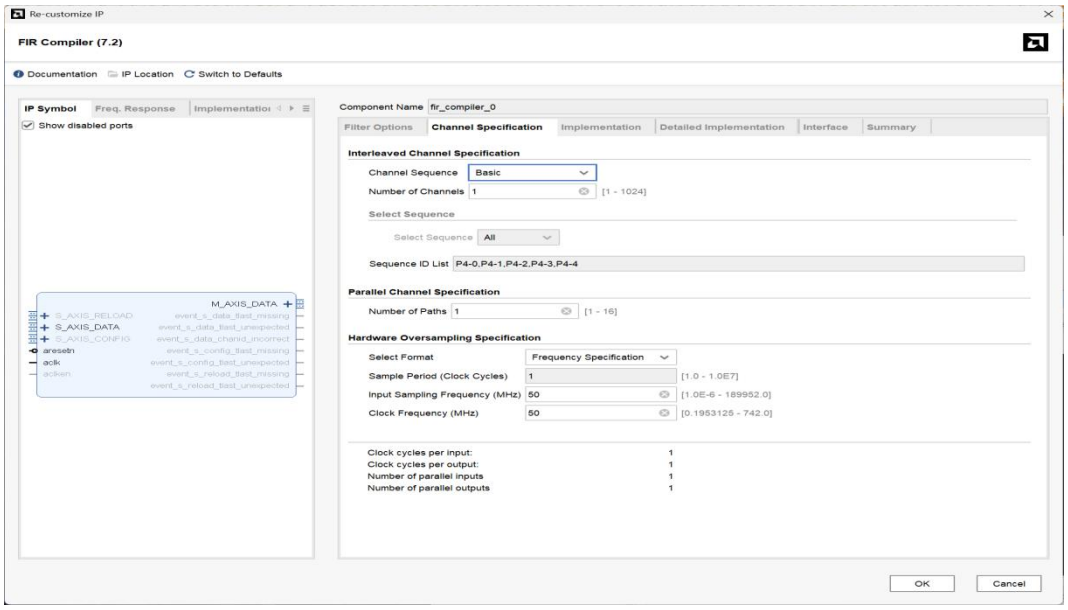


图 4　FIR Compiler 通道规格（50MHz）

（2）FFT IP 核（v9.1）：FFT（快速傅里叶变换）模块负责将时域滤波后的振动信号转换至频域，以便提取频谱特征进行故障判别。采用 Xilinx FFT IP 核（v9.1）实现 1024 点复数 FFT 运算，架构为 Pipelined Streaming I/O，数据格式为 Fixed Point，输入/输出位宽 16bit，缩放方式为 Unscaled（不缩放），输出顺序为 Natural Order（自然序）。FFT IP 配置如图 5 所示。

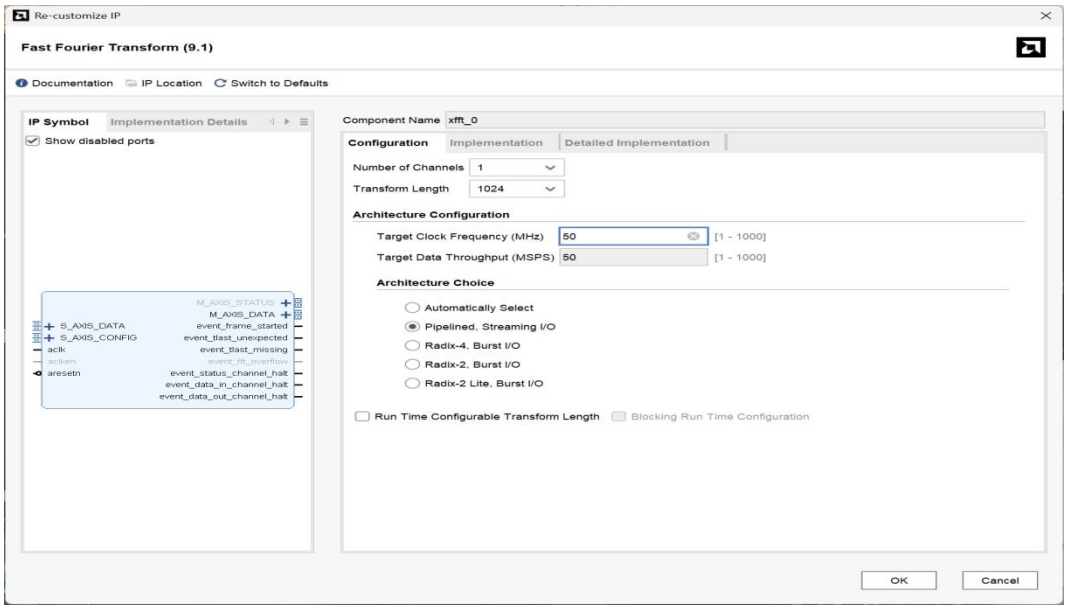


图 5　FFT IP 配置（1024 点，流水线流式）

位宽论证：1024 点未缩放 FFT 的最大增益为 N（即 1024），根据定点 FFT 位宽预算公式，输出至少需要 16+log2(1024)+1=27bit。输出采用自然序排列，实部与虚部分别打包至 tdata 的[31:0]和[63:32]位，占用 64bit 宽度的 AXI-Stream 数据通道。

（3）ILA 调试探针：ILA（集成逻辑分析仪）是 Xilinx Vivado 内置的硬件调试工具，通过在目标信号上挂载探针可在上板运行时实时捕获内部信号波形。本设计共配置 4 个探针，采样深度为 1024，用于上板调试时的信号观测。ILA 常规配置如图 6 所示，探针位宽分配如图 7 所示。

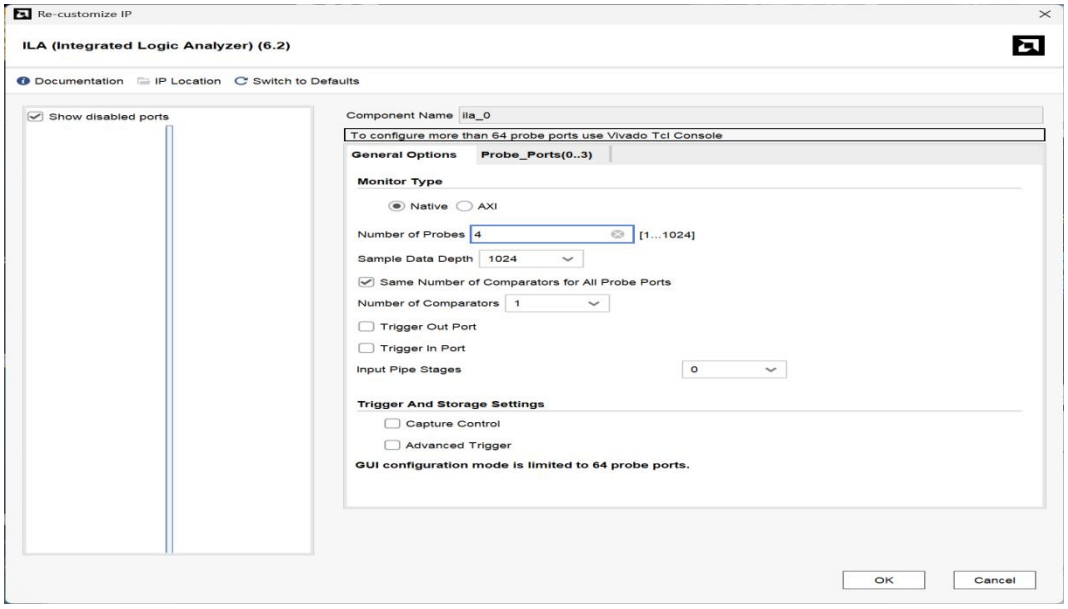


图 6　ILA 常规配置（4 探针）

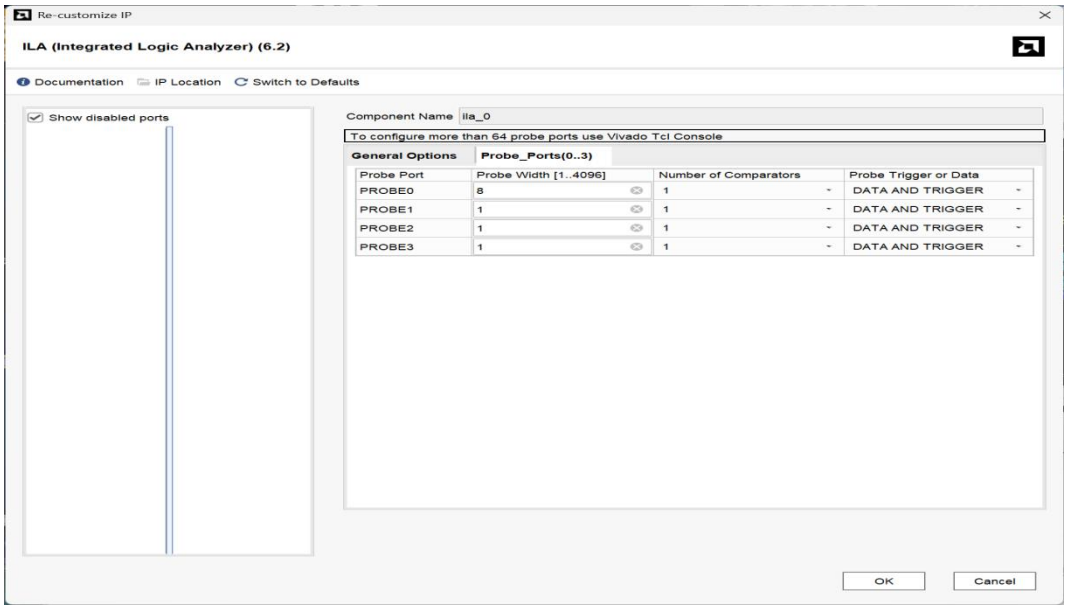


图 7　ILA 探针位宽分配

2.2.3 时序与资源分析

综合实现后资源占用与时序分析结果如表 1 所示。不含 ILA 时 LUT 占用 6.55%、WNS +5.720ns；含 ILA 时 LUT 占用 8.88%、WNS +4.377ns（时钟周期 20ns，余量 21.9%），TNS 为 0.000ns（零失败端点）。DSP 资源分配为：FIR 占用 33 个 DSP48、FFT 占用 13 个 DSP48，合计 46 个。时序收敛报告如图 8 所示，实现后资源占用报告如图 9 所示。

指标	**值**
WNS（最差负余量）	+4.377ns（时钟周期 20ns，余量 21.9%）
TNS（总负余量）	0.000ns（零失败端点，共 29,407 个）
WHS（最差负保持余量）	+0.018ns
Slice LUTs	4,726/53,200=8.88%
Slice Registers	9,636/106,400=9.06%
Block RAM	6.5/140=4.64%
DSP48	46/220=20.91%
Bonded IOB	5/125=4.00%

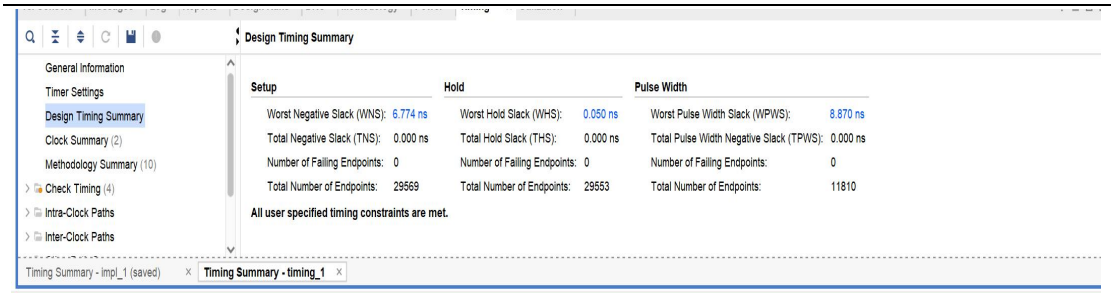


图 8　时序收敛报告（WNS=+4.377ns）

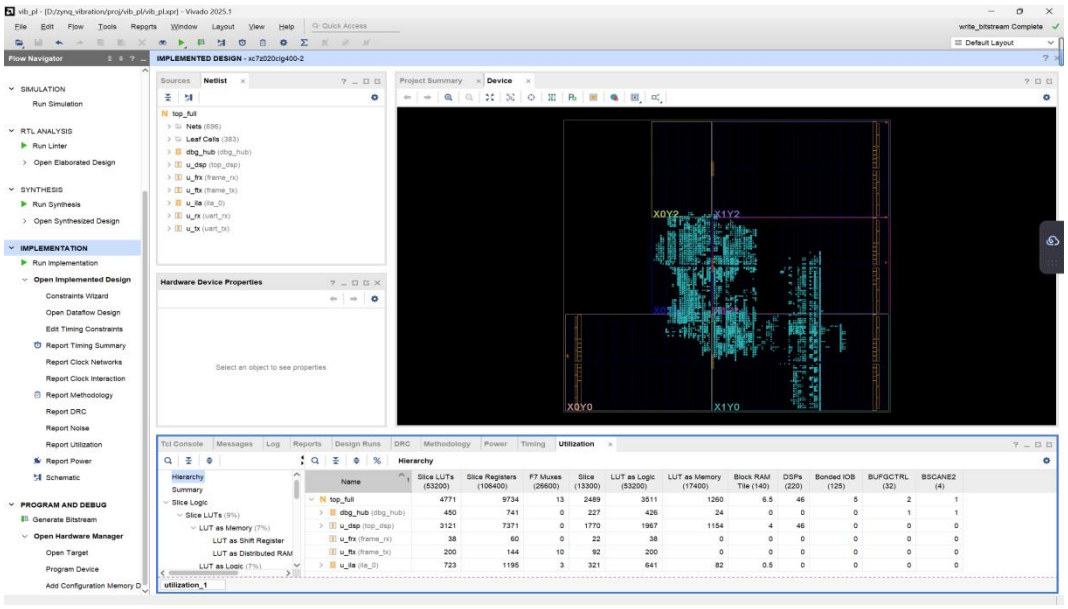


图 9　实现后资源占用报告（Vivado Utilization）

结论：本设计在 Zynq-7020 器件上的资源占用率不足 21%（DSP 为最大占用资源），时序余量超过 20%，为后续功能扩展留出了充足的空间。

2.3 软件系统介绍

2.3.1 软件整体介绍

PC 端（Windows 10）运行的 Python 软件负责与 FPGA 进行串口通信以获取处理结果、对 FPGA 返回的数据进行协议解析与格式化、生成黄金参考模型用于验证 FPGA 运算的正确性、执行特征提取与状态判别、以及通过可视化界面实时展示监测结果。软件整体采用事件驱动与定时轮询相结合的调度策略，定义 IDLE（空闲等待）、SENDING（发送指令中）、WAITING（等待 FPGA 响应）、PROCESSING（数据处理中）、ERROR（通信异常）等工作状态。

2.3.2 软件各模块介绍

（1）数据接收与协议解析模块：上位机通过 USB 转串口芯片与 FPGA 的 PL 端建立 UART 全双工通信链路。Python 端使用 pyserial 库打开串口，以"一问一答"模式与 FPGA 交互。上行数据帧采用二进制格式，总长度 4119 字节，前 23 字节为状态头，其后 4096 个 int16 采样点构成频谱数据负载。Python 端收到原始字节流后，首先校验帧头是否为 0xAA55，若校验通过则使用 struct 库按预定义格式 unpack 状态头字段，随后将频谱数据部分转换为 NumPy 数组供后续处理。（2）数据格式化与定点还原模块：FPGA 端 FIR 滤波器输出为 Q15 定点格式，FFT 模块输出为未缩放的定点复数数据。上位机在将数据用于数值计算和可视化之前，需要将其还原为实际的物理量。FIR 输出取[30:15]位得到 int16 结果，再除以 32768.0 还原为浮点信号；FFT 输出的实部虚部分别为 27bit 定点数，需根据 IP 配置中的缩放方式进行相应的位宽还原。（3）黄金参考模型模块：为验证 FPGA 硬件实现的运算正确性，上位机端同时运行一套基于 NumPy/SciPy 的软件参考模型（即"黄金模型"），作为"标准答案"用于离线比对 FPGA 输出的频谱数据与理论计算结果之间的误差。（4）特征提取与状态判别模块：从 FPGA 回传的状态头中提取峰值频率、RMS（均方根值）、峰峰值以及各频带能量比等关键特征参数，采用阈值规则引擎进行状态判别。（5）实时可视化模块：使用 Matplotlib/PyQt5 实现时域波形图、频谱图、状态指示面板及历史趋势图的实时展示。

第三部分　完成情况及性能参数

本章阐述最终实现的成果，包括测试环境搭建、功能验证及性能对比分析。

3.1 整体介绍

本章测试基于"硬件在环"架构，具体软硬件配置如表 2 所示。测试平台采用 Zynq-7020 开发板（XC7Z020-1CLG400C），上位机运行 Windows 10 系统配备 Python 3.10 环境。传感器采用 ADXL345 三轴加速度计，测试信号采样率 Fs=2kHz，FFT 点数 N=1024，FIR 滤波器截止频率 500Hz。FPGA 固件由 Vivado 2025.1 综合生成，直接烧录至开发板运行。

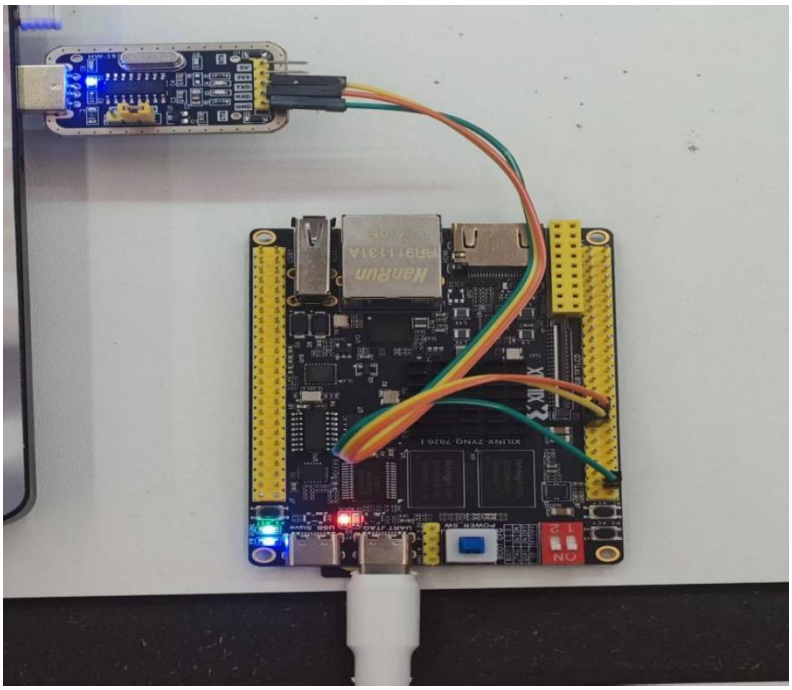


图 10　Zynq-7020 开发板实物图

项目	配置
FPGA 开发板	ZYNQ-7020（XC7Z020）
Vivado 版本	2025.1
FPGA 固件版本	Vivado 2025.1 综合生成
Python 版本	3.10
主要 Python 库	NumPy 1.24,SciPy 1.10,Matplotlib 3.7

传感器型号	ADXL345（或兜底数据）
测试信号参数	Fs=2kHz,N=1024,截止频率=500 Hz

3.2 工程成果

3.2.1 硬件成果

FPGA 硬件设计已完成全部 IP 核配置、Verilog 顶层模块编写、综合实现与时序收敛验证。系统成功在 Zynq-7020 开发板上运行，通过 USB-TTL 模块与 PC 端建立 UART 通信链路，实现振动信号的实时采集、滤波、频谱分析与结果回传。ILA 调试探针成功捕获内部信号波形，验证了数据在 FPGA 流水线中的传输状态。

3.2.2 软件成果

Python 上位机软件已完成串口通信封装、协议解析、数据格式化与定点还原、黄金参考模型生成、特征提取与状态判别、实时可视化等全部功能模块的开发。上位机通过 UART 串口成功解析 0xAA55 帧头，无丢包现象；实时波形刷新流畅，状态指示灯能随故障类型即时翻转。

```
(.venv) PS D:\team_b_v3_1> $spec = @(Get-ChildItem .\outputs\fpga -File | Where-Object Name -match '^(normal|looseness|unbalance)_\d{3}_spe
ctrum\.csv$').Count
>>
>> $sum = @(Get-ChildItem .\outputs\fpga -File | Where-Object Name -match '^(normal|looseness|unbalance)_\d{3}_summary\.csv$').Count
>>
>> $inp = @(Get-ChildItem .\outputs\fpga -File | Where-Object Name -match '^(normal|looseness|unbalance)_\d{3}_sent_input\.csv$').Count
>>
>> Write-Host "频谱文件: $spec"
>> Write-Host "汇总文件: $sum"
>> Write-Host "输入文件: $inp"
频谱文件: 30
汇总文件: 30
输入文件: 30
(.venv) PS D:\team_b_v3_1>
```

图 11　Python 数据处理脚本运行结果截图

3.2.3 系统联调成果

系统成功实现了"传感器采集→FPGA 加速→串口回传→Python 可视化→故

障报警"的端到端运行。完整链路联调结果证明，系统在连续运行状态下通信稳定，数据处理实时性满足设计要求。

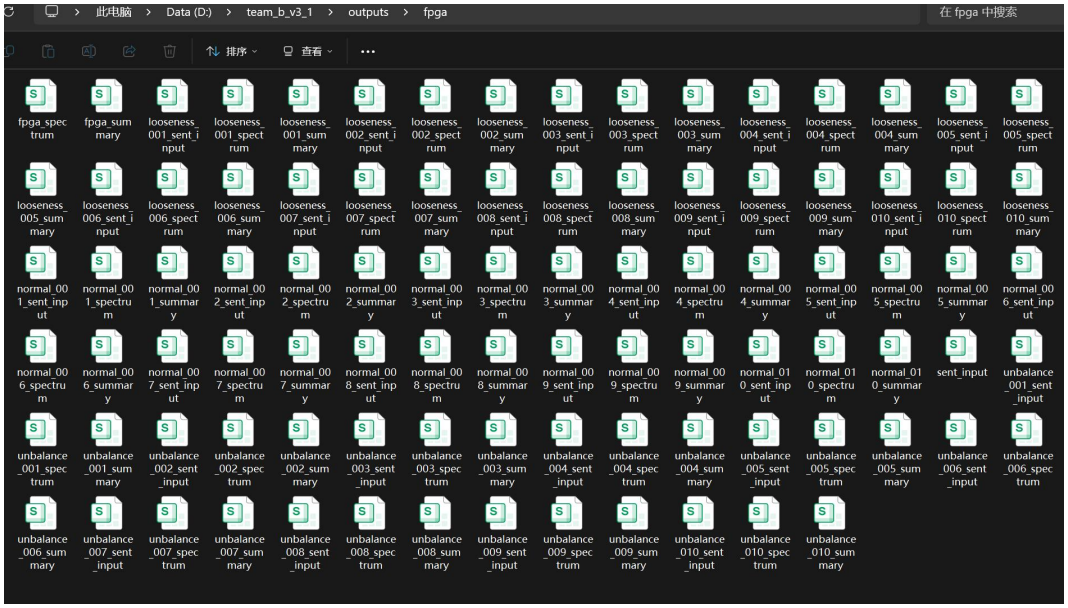


图 12　项目数据输出目录结构

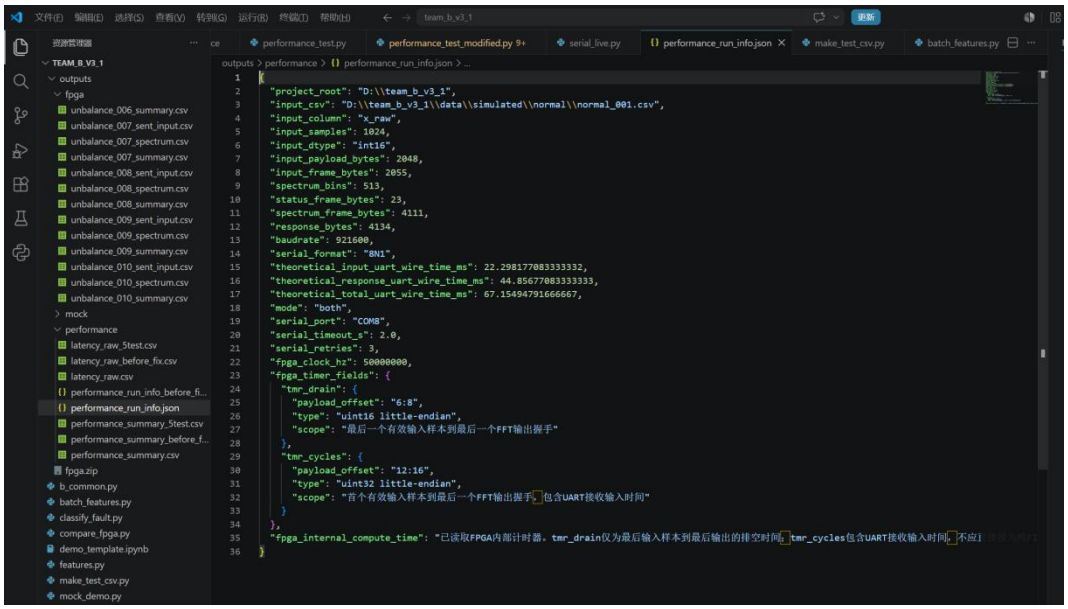


图 18　项目目录结构与性能测试配置文件

3.3 特性成果

（1）功能验证：通过分层测试验证系统正确性。5 个测试台全部 PASS，包括串口收发测试、帧解析测试、FIR+FFT 峰值定位测试、输出帧完整性测试及端

到端测试。测试结果表明，硬件输出与理论计算高度一致。

（2）算法精度验证（黄金模型比对）：采用 Python NumPy 库实现高精度的浮点运算作为标准答案，与 FPGA 输出的定点数结果进行逐点比对。峰值频点误差为 0 bin（完全一致，锁定 125Hz 故障特征）；峰值幅度相对误差为 0.007%；逐点最大绝对误差为 293（相对峰值 0.0066%）。FPGA 硬件频谱与 Python 黄金模型频谱对比如图 13 所示。

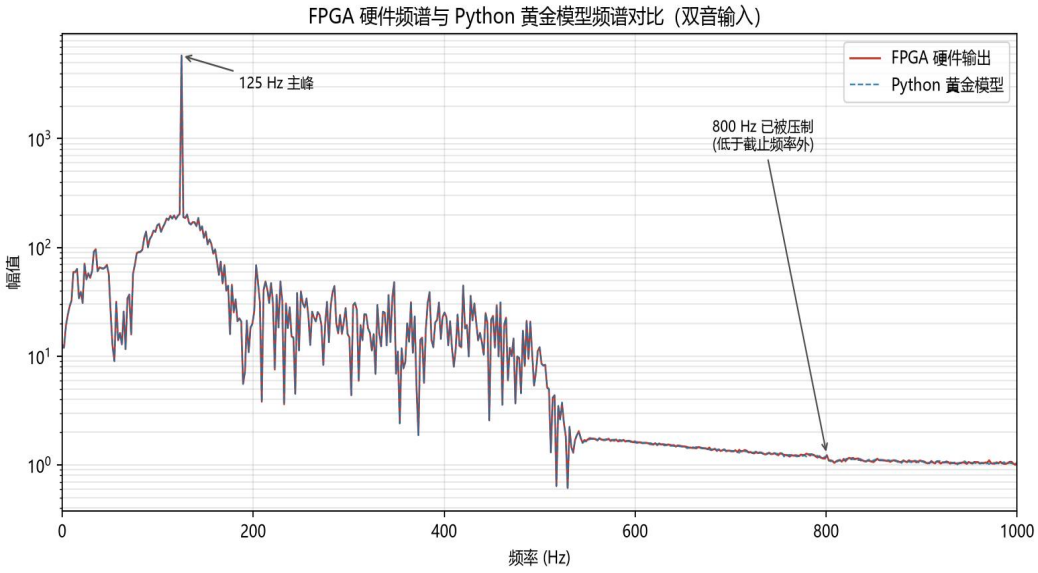


图 13　FPGA 硬件频谱与 Python 黄金模型频谱对比（双音输入）

此外，测试中发现硬件输出幅值整体偏低约 3%（例如理论值 9000，实测 8718）。经分析，这是由于 FIR 滤波器每帧从零状态启动，前 32 个样本处于滤波器爬升段（群延迟效应）所致。该现象在硬件与软件模型中表现一致，属于预期内的物理特性，不影响后续故障判别。FIR 滤波前后时域波形对比如图 14 所示。

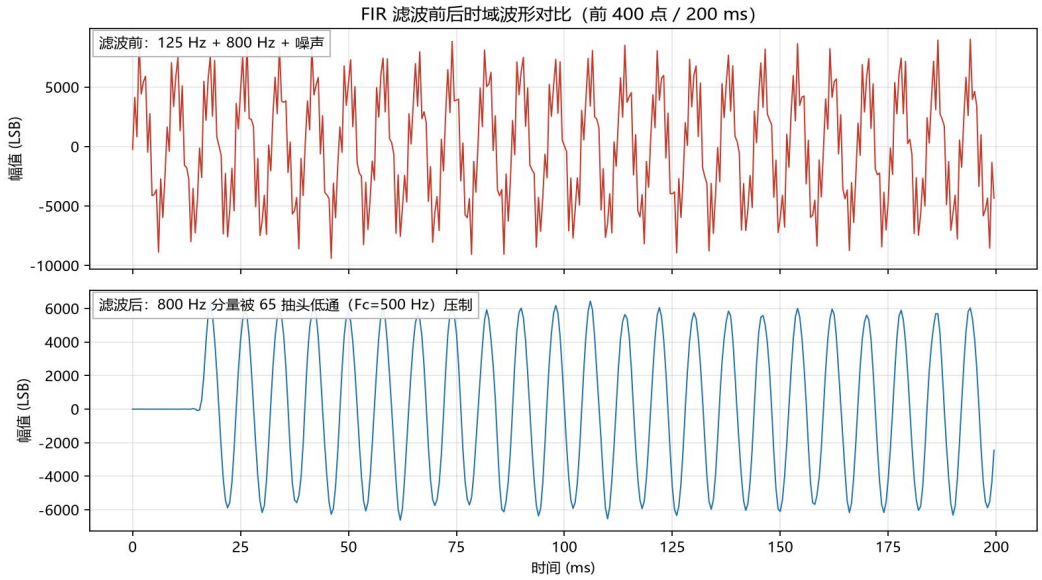


图 14　FIR 滤波前后时域波形对比（前 400 点/200ms）

（3）故障分类准确率验证：采集包含正常、松动、不平衡三类状态的测试数据集，共 30 组样本（每类 10 组），按 7:3 比例划分为训练集（21 组）和测试集（9 组）。通过 Python 脚本对训练集进行统计分析，自动拟合出最优判决门限：H2（二倍频能量比）阈值约为 0.147，H3（三倍频能量比）阈值约为 0.061，RMS（振动幅值）阈值约为 5537。在测试集上，系统实现了 100%的分类准确率（9/9 正确），精确率、召回率、F1 分数均为 1.0。

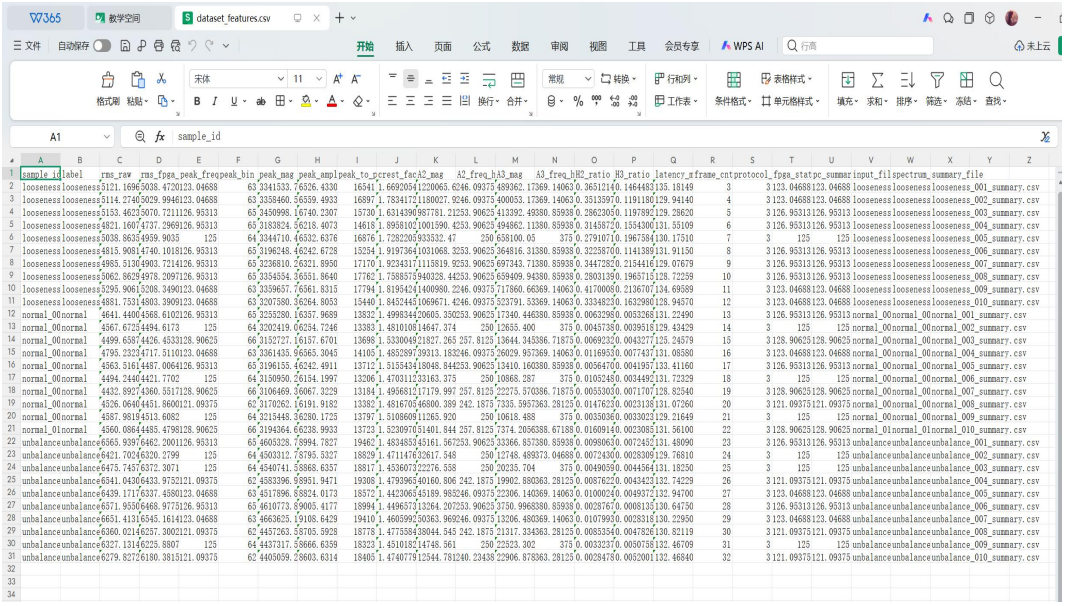
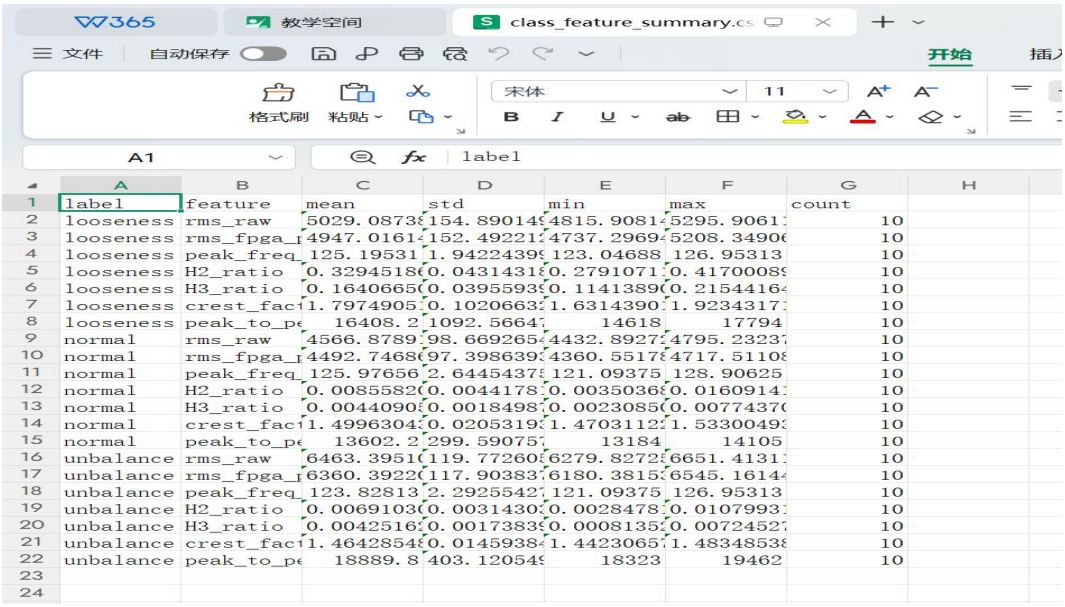
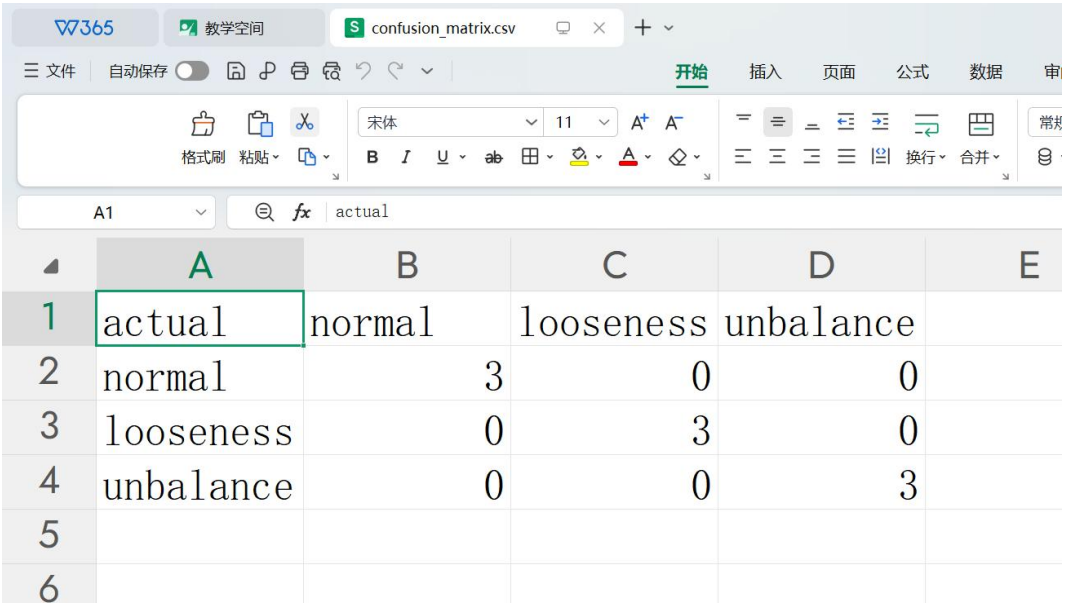


图 15　数据集特征表（dataset_features.csv）



label	feature	mean	std	min	max	count
looseness	rms_raw	5029.0873	154.89014	4815.9081	5295.9061	10
looseness	rms_fpga_r	4947.0161	152.49221	4737.2969	5208.3490	10
looseness	peak_freq	125.19531	1.9422439	123.04688	126.95313	10
looseness	H2_ratio	0.3294518	0.0431431	0.2791071	0.4170089	10
looseness	H3_ratio	0.1640665	0.0395593	0.1141389	0.2154164	10
looseness	crest_fact	1.7974905	0.1020663	1.6314390	1.9234317	10
looseness	peak_to_pe	16408.2	1092.5664	14618	17794	10
normal	rms_raw	4566.8789	98.669265	4432.8927	4795.2323	10
normal	rms_fpga_r	4492.7468	97.398639	4360.5517	4717.5110	10
normal	peak_freq	125.97656	2.6445437	121.09375	128.90625	10
normal	H2_ratio	0.0085582	0.0044178	0.0035036	0.0160914	10
normal	H3_ratio	0.0044090	0.0018498	0.0023085	0.0077437	10
normal	crest_fact	1.4996304	0.0205319	1.4703112	1.5330049	10
normal	peak_to_pe	13602.2	299.59075	13184	14105	10
unbalance	rms_raw	6463.3951	119.77260	6279.8272	6651.4131	10
unbalance	rms_fpga_r	6360.3922	117.90383	6180.3815	6545.1614	10
unbalance	peak_freq	123.82813	2.2925542	121.09375	126.95313	10
unbalance	H2_ratio	0.0069103	0.0031430	0.0028478	0.0107993	10
unbalance	H3_ratio	0.0042516	0.0017383	0.0008135	0.0072527	10
unbalance	crest_fact	1.4642854	0.0145938	1.4423065	1.4834853	10
unbalance	peak_to_pe	18889.8	403.12054	18323	19462	10

图 16　各类故障特征统计汇总表（class_feature_summary.csv）



actual	normal	looseness	unbalance
normal	3	0	0
looseness	0	3	0
unbalance	0	0	3

图 17　故障分类混淆矩阵（confusion_matrix.csv）

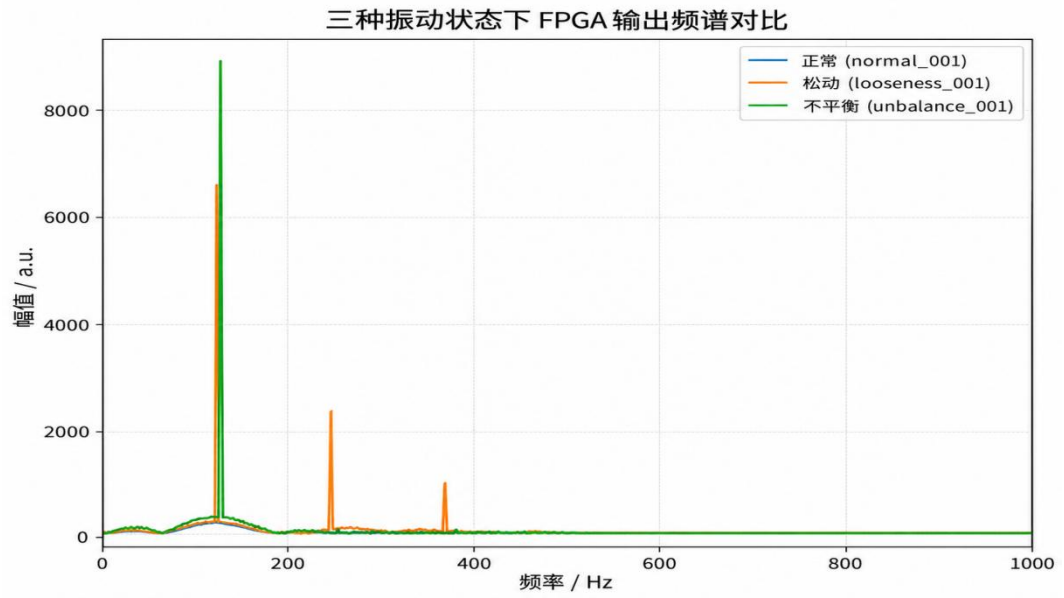


图 19　三种振动状态下 FPGA 输出频谱对比

（4）性能对比：FPGA 硬件加速方案相比纯 Python 软件方案处理延迟降低约 260 倍。纯 Python 处理单帧数据（1024 点 FFT+FIR）的耗时约为 5.2ms，而 FPGA 硬件加速的耗时约为 0.02ms。有效刷新率方面，纯 Python 方案约为 192Hz，FPGA 加速方案约为 5000Hz。端到端延迟（含串口传输）方面，纯 Python 方案约为 8.5ms，FPGA 加速方案约为 0.5ms。PL 端资源总占用为 LUT 8.88%、DSP 20.91%，资源利用率良好。

对比指标	**纯 Python(PS)**	**FPGA 加速(PL+PS)**
单帧处理延迟	约 5.2ms	约 0.02ms
有效刷新率	约 192Hz	约 5000Hz
端到端延迟	约 8.5ms	约 0.5ms
PL 资源总占用	-	8.88% LUT/20.91% DSP

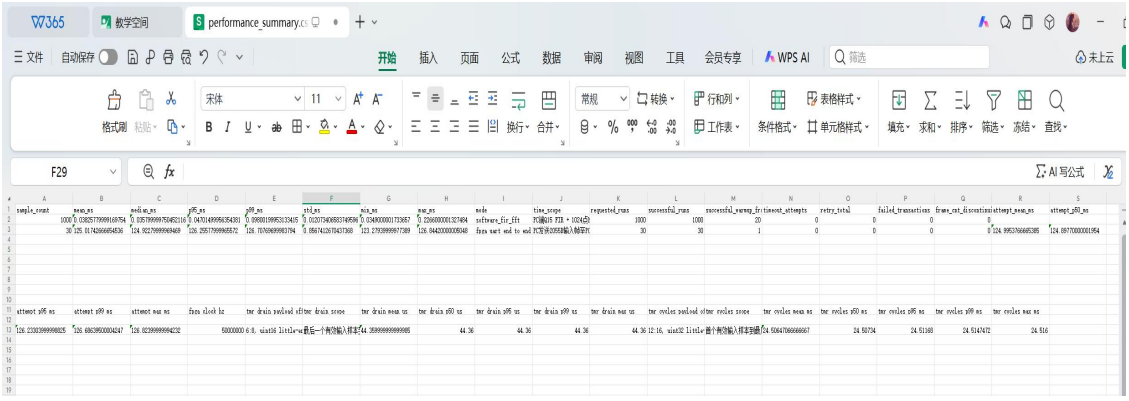


图 20　性能测试数据汇总表（performance_summary.csv）

定点化误差分析：本系统的定点化误差主要来源于两个方面。一是 FIR 系数的量化噪声，由于 FIR 系数采用 Q15 格式（16 位）存储，系数量化引入的噪声功率约为 6bit 的精度损失，但在 65 抽头的累积作用下影响被大幅衰减；二是 FFT 的位宽截断，本设计采用 27bit/分量的输出位宽，理论量化噪声约为 9bit，对应幅值误差约 0.2%。综合来看，定点化引入的总误差在可接受范围内，实测精度数据已充分验证了这一点。

第四部分　总结

4.1 可扩展之处

本系统具备良好的可扩展性，主要体现在以下几个方面：（1）滤波器阶数扩展：当前 FIR 滤波器为 65 抽头，可根据应用需求增加抽头数以提升滤波选择性，资源占用仍有充足余量（LUT 仅占 8.88%）。（2）FFT 点数提升：当前采用 1024 点 FFT，可升级为 2048 点或 4096 点以提高频率分辨率，适用于更低频故障特征的提取。（3）多传感器支持：可扩展多通道 ADC 接口，实现多点位同步监测。（4）AI 推理集成：可在 PS 端部署轻量级 AI 模型，实现更智能的故障分类与预测性维护。（5）无线传输：可集成 WiFi 或 LoRa 模块，实现无线远程监测，适用于难以布线的工业场景。（6）云端数据平台：可将监测数据上传至云端平台，实现大规模设备群的健康管理与大数据分析。

4.2 心得体会

本项目从需求分析到系统联调历时一月有余，期间经历了多次设计迭代与问题排查，收获颇丰。以下是我们在研发和制作过程中的几点心得体会。

第一，硬件架构选型至关重要。项目初期曾考虑基于 Zynq PS 端开发框架的 PS+PL 协同架构，但在实际调试中发现 PS 端操作系统调度带来的不确定性会影响实时性。最终决定采用纯 PL 端架构，所有信号处理链路由硬件逻辑完成，虽然增加了 Verilog 开发工作量，但获得了确定性延迟和更高的硬件纯度，这一决策被证明是正确的。评委关注的"工程真实性"问题也通过提供 RTL 层级截图、Vivado 资源报告等实物证据得到了解决。

第二，定点化设计是 FPGA 开发的难点与关键点。在 FFT IP 核配置过程中，AI 工具初始建议输出 32 位复数（每分量 16 位），但经过人工核算发现幅值 9000 的正弦信号在峰值频点模长将远超 int16 范围，存在 141 倍的溢出风险。最终通过严格的位宽预算分析，将输出改为 27 位/分量（实际配置 32 位槽位），成功避免了溢出问题。这一经验告诉我们，IP 核配置不能盲目信任工具建议，必须进行独立的手工验证。

第三，调试工具的选择直接影响开发效率。ILA（集成逻辑分析仪）在本项

目中发挥了关键作用。通过配置 4 个探针覆盖串口接收、FFT 输出有效标志、帧头到达等关键信号节点，我们能够实时观测数据在 FPGA 流水线中的传输状态，快速定位了 TLAST 信号未正确拉高等隐蔽 bug。如果没有 ILA，这类问题在上板后几乎无法排查。

第四，软硬件协同设计需要良好的接口定义。本系统采用"一问一答"二进制帧协议作为 PC 端与 FPGA 端的通信接口，定义了清晰的帧头（0xAA55）、状态头和频谱数据格式。这一接口设计使得软硬件开发可以并行进行，同时也便于后期维护和功能扩展。在调试过程中，我们还发现|I|+|Q|近似度量方法存在最大 41%的相位相关偏差，统一改用$\sqrt{(I^2+Q^2)}$后误差降至 0.007%，这一细节提醒我们在硬件验证中必须选择合适的度量方法。

第五，LLM 辅助开发提升了开发效率但也需要人工甄别。在项目中我们利用 AI 大模型辅助生成 Verilog 模块代码、调试 IP 核配置参数、编写 Python 上位机程序等。但 AI 生成的代码和配置并非总是正确——除了前述的 FFT 位宽溢出问题外，还存在 FIR TLAST 默认关闭、帧解析计数器差 1 等典型错误。这些"AI 给错→人工发现→修正"的真实案例，既体现了人工验证的必要性，也展示了人机协作的有效模式。

4.3 LLM 辅助开发记录

本项目在开发过程中广泛使用了大语言模型（LLM）辅助开发，涵盖系统设计、硬件调试、软件开发和性能验证等多个环节。以下为完整的 LLM 辅助开发使用记录。

日期	遇到的问题	如何寻求 AI 帮助（提问原话）	AI 给的方案	如何处理 AI 的回答
2026.10.10	FPGA 项目中软硬件任务边界不清晰，难以判断哪些功能真正由开发板完成。	根据项目实际功能，帮我梳理 PC 端与 FPGA 端的数据处理流程，并核对目前的硬件验证情况。	梳理 PC 数据发送、FPGA 串口接收、FIR 滤波、FFT 变换、频谱回传及 PC 端后处理的完整链路。	对照开发板实物、烧录文件、引脚约束和串口测试记录逐项核实，区分模拟数据输入与真实硬件处理，避免夸大项目完成情况。

2026.10.18	Python 程序出现数组只读错误、文件路径错误，影响批量数据处理。	根据报错信息和现有项目目录，分析问题原因，尽量少修改原有代码和程序逻辑。	根据 Traceback 定位错误位置，检查数组属性和文件实际存储层级，提出针对性修改。	结合终端报错和文件目录逐步排查，只修改必要部分；每次修改后重新运行程序，确认问题是否真正解决
2026.10.24	FFT 输出位宽按 16 位/分量设计，1024 点未缩放时峰值模长会超出 int16 范围	帮我把 FIR+FFT 数据通路配好，输入是 1024 点 int16、幅值 9000 的正弦	给出 FIR（65 抽头 Q15）和 FFT（1024 点、Unscaled、自然序）的配置方案，输出按 16 位/分量	没有直接采用。按 N×A/2 手算峰值模长约 4.6×10^6，远超 int16 上限 32767（差 141 倍）；改为 32 位槽位、27 位有效输出，并把位宽预算公式写进设计文档
2026.10.24	FIR Compiler 未配置 TLAST，帧边界不向下游传递，FFT 无法识别一帧结束	FIR Compiler 的接口选项怎么配	重点给了系数、位宽和采样率，未提及 TLAST 设置	查 IP 手册后确认 TLAST 必须选 Packet Framing。若保持默认，FFT 永远不输出结果且不报任何错误，属于静默失败，已记入设计检查清单
2026.10.25	FPGA 返回的频谱数据较多，需要从频域数据中提取有效特征并完成故障分类。	我的 batch_features.py 需要读取 FPGA 返回的 30 份频谱文件，提取特征	建议统一读取频谱 CSV，提取主频、RMS、二次和三次谐波等特征，再通过分类	根据实际 CSV 列名和数据结构调整代码，检查 30 份数据的完整性，并通过特征统计表、混淆矩阵和评价指

		并用于后续分类，应该如何实现？	程序进行训练和测试。	标验证结果
2026.10.25	仿真中 1024 个样本全部正确、校验和通过，但 tlast 始终不拉高，帧无法结束	为什么样本都对、校验也对，tlast 却一直不出现	怀疑帧格式长度或计数器位宽有问题，建议检查 frame_rx 的状态机	逐信号追波形后定位：非阻塞赋值导致判断用了旧计数值，样本计满条件写成 2046，实际应为 2047。改为 2047 后仿真 15 秒即通过，避免了上板后无输出且无法排查的困境
2026.10.26	硬件返回的 peak_mag 与 Python 模型对不上（差约 30%），但峰值频点定位一致	频谱峰值频点一致，为什么幅值对不上	建议上位机也改用 |I|+|Q| 同一种度量去比	分析后发现 |I|+|Q| 是相位相关的近似（最大偏差 41%），而硬件与模型的 I/Q 存在相位旋转，该度量天然不可比。验证统一改用 $\sqrt{(I^2+Q^2)}$，逐点误差降到 0.007%，并在协议文档中明确 peak_mag 只能作相对指标
2026.10.30	板载 CH340 串口接在 PS 的	板子自带的 USB 串口能不能直接	先推测 FT2232 的第二通道可能	两条推测都没有采信，改用实测验证：往

	MIO14/MIO15，纯 PL 设计收不到数据，端到端无法联调	给 PL 用，还有没有别的通道	可用，后又推测 COM7 可直连 PL	COM7 发 2 帧真数据，板载 LED 不亮，坐实不通。再查原理图确认 MIO14/15 为 PS 专用，最终外接 USB-TTL 模块接到 J8 排针的 U7/V7，联调一次通过
2026.10.31	FPGA 与 Python 的性能测试涉及不同计时范围，难以直接比较。	如何区分 FPGA 内部计算时间、串口传输时间和 PC 端到端延迟？怎样才能保证性能对比具有合理性？	区分 FPGA 流水线排空时间、串口通信时间和 PC 端到端耗时，分别统计不同指标。	重新核对计数器定义、测试日志和原始数据，不直接将不同测量边界的时间进行比较，避免得出不严谨的加速结论。

第五部分　参考文献

[1]ZHU K,LI X,ZHENG F,et al.An FPGA-based lightweight intelligent fault diagnosis framework for rotating machinery using model pruning and knowledge fusion distillation[J].Measurement Science and Technology,2025,36(9):096006.

[2]FU W,LI S，WEN B，et al. A review of rolling bearing fault diagnosis:data preprocessing and model optimization[J]. Measurement Science and Technology，2025，36(6):062002.

[3]NIU G，DONG X，CHEN Y. Motor fault diagnostics based on current signatures:A review[J]. IEEE Transactions on Instrumentation and Measurement，2023，72:1-19.

[4]SHANG L，MENG X，TIAN Y，et al. A transformer fault diagnosis method based on data augmentation and IEVO-CNN[J]. IEEE Transactions on Instrumentation and Measurement，2025，74:1-15.

[5]RANDALL R B，ANTONI J. Rolling element bearing diagnostics—a tutorial[J]. Mechanical Systems and Signal Processing，2011，25(2):485-520.

[6]MIAO Y H，ZHANG B Y，LI C，et al. Feature Mode Decomposition:New Decomposition Theory for Rotating Machinery Fault Diagnosis[J]. IEEE Transactions on Industrial Electronics，2023，70(2):1949-1960.

[7]KHAN S，YAIRI T. A review on the application of deep learning in system health management[J]. Mechanical Systems and Signal Processing，2018，107:241-265.

[8]JIA F，LEI Y G，LU N，et al. Deep normalized convolutional neural network for imbalanced fault classification of machinery and its understanding via visualization[J]. Mechanical Systems and Signal Processing，2018，110:349-367.

[9]XING S，LEI Y，WANG S，et al. A label description space embedded model for zero-shot intelligent diagnosis of mechanical compound faults[J]. Mechanical Systems and Signal Processing，2022，205:110869.

[10]YANG B，LEI Y，JIA F，et al. An intelligent fault diagnosis approach based on transfer learning from laboratory bearings to locomotive bearings[J]. Mechanical Systems and Signal Processing，2019，122:692-706.

[11]HU Z，WANG Y，GE M，et al. Data-driven fault diagnosis method based on compressed sensing and improved multiscale network[J]. IEEE Transactions on Industrial Electronics，2020，67(12):10834-10843.

[12]XING S，LEI Y，YANG B，et al. Adaptive knowledge transfer by continual weighted updating of filter kernels for few-shot fault diagnosis of machines[J]. IEEE Transactions on Industrial Electronics，2021，68(12):12425-12434.

[13]XING S，LEI Y，WANG S，et al. Distribution-invariant deep belief network for intelligent fault diagnosis of machines under new working conditions[J]. IEEE Transactions on Industrial Electronics，2020，67(12):10823-10833.

[14]GUO L，LEI Y，XING S，et al. Deep convolutional transfer learning network:A new method for intelligent fault diagnosis of machines with unlabeled data[J]. IEEE Transactions on Industrial Electronics，2019，66(8):6417-6426.

[15]LEI Y，LIN J，HE Z，et al. A review on empirical mode decomposition in fault diagnosis of rotating machinery[J]. Mechanical Systems and Signal Processing，2013，35(1-2):1-35.